

向量检索深度解析:从暴力搜索到IVF与HNSW前沿算法

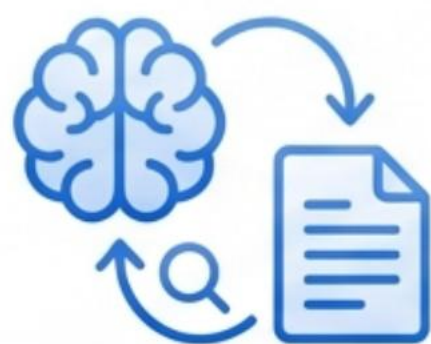


向量嵌入(Vector Embeddings) | 近似最近邻(ANN) | 倒排文件索引(IVF) | 分层导航小世界(HNSW)

向量数据库检索算法

向量搜索的背景与意义

- ✓ 随着深度学习的发展，向量（Embedding）在图像、文本、语音等领域的应用越来越广泛。
- ✓ 向量搜索（Vector Search）成为在大规模数据中查找相似向量的核心技术。



检索增强生成(RAG)

为大语言模型(LLM)从庞大的知识库中检索最相关的内容，以生成更准确的回答。



推荐系统 (Recommendation Systems)

根据用户的向量画像，在海量商品或内容库中找到最相似的推荐项。



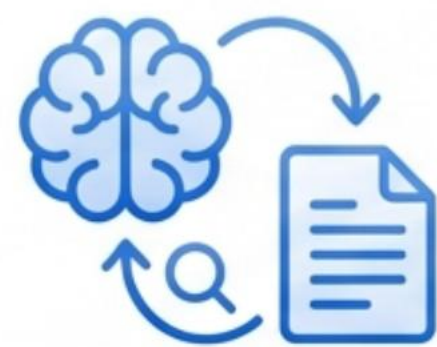
图像/视频检索(Image/Video Search)

实现“以图搜图”，根据图像的视觉特征向量找到相似的图片。

向量数据库检索算法

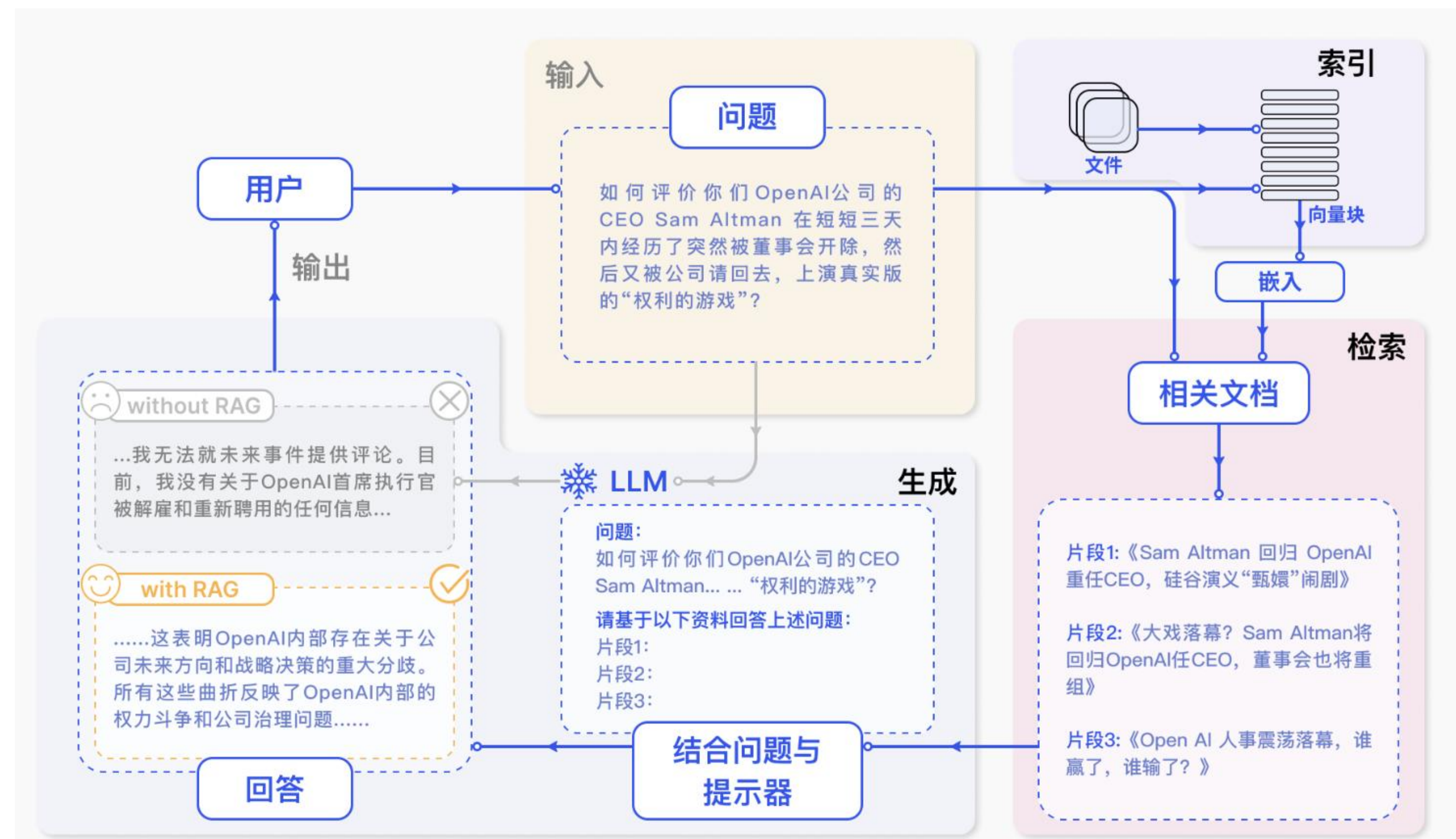
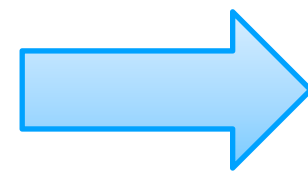
向量搜索的背景与意义

- ✓ 随着深度学习的发展，向量（Embedding）在图像、文本、语音等领域的应用越来越广泛。
- ✓ 向量搜索（Vector Search）成为在大规模数据中查找相似向量的核心技术。



检索增强生成(RAG)

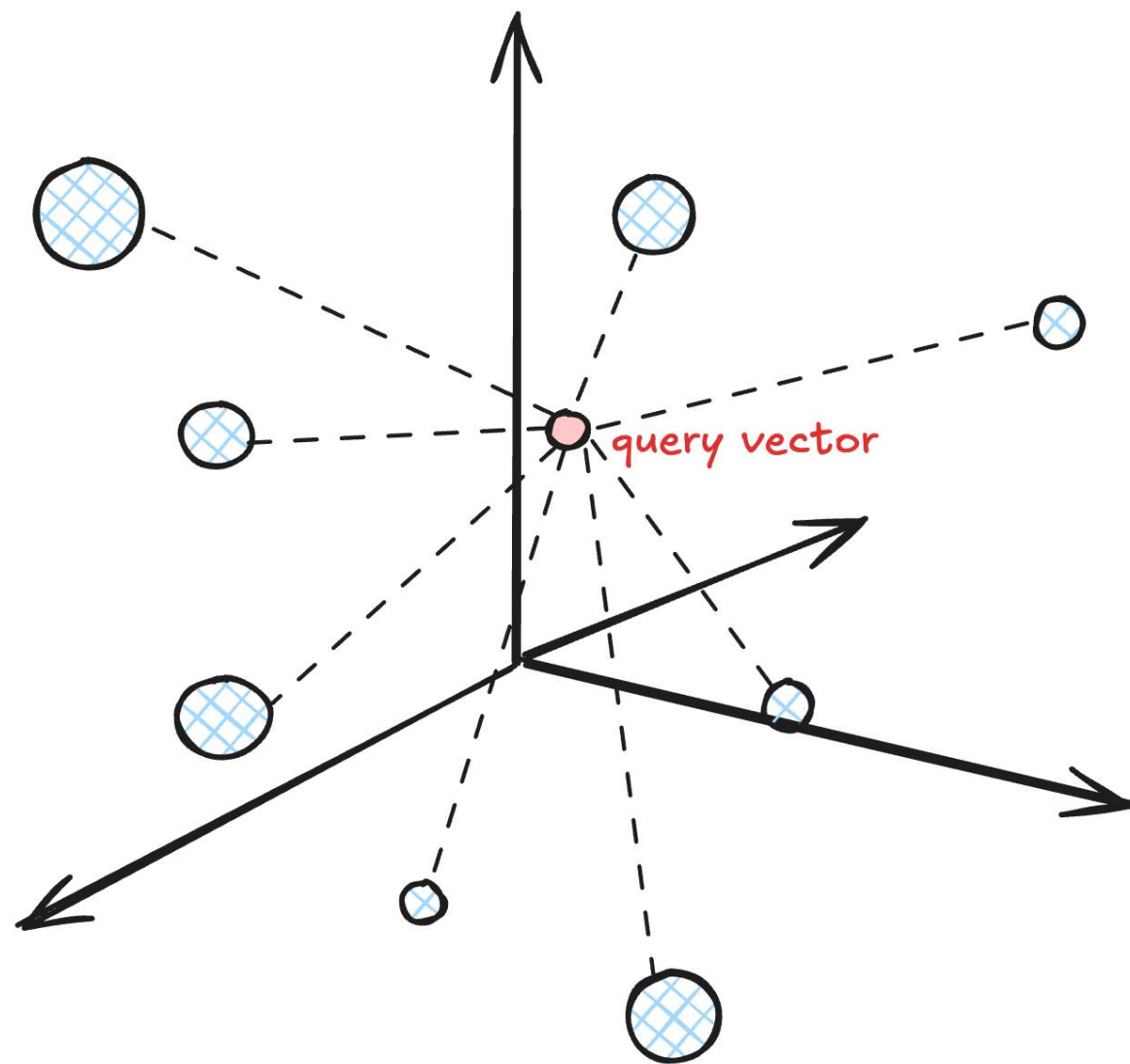
为大语言模型(LLM)从庞大的知识库中检索最相关的内容，以生成更准确的回答。



向量数据库检索算法

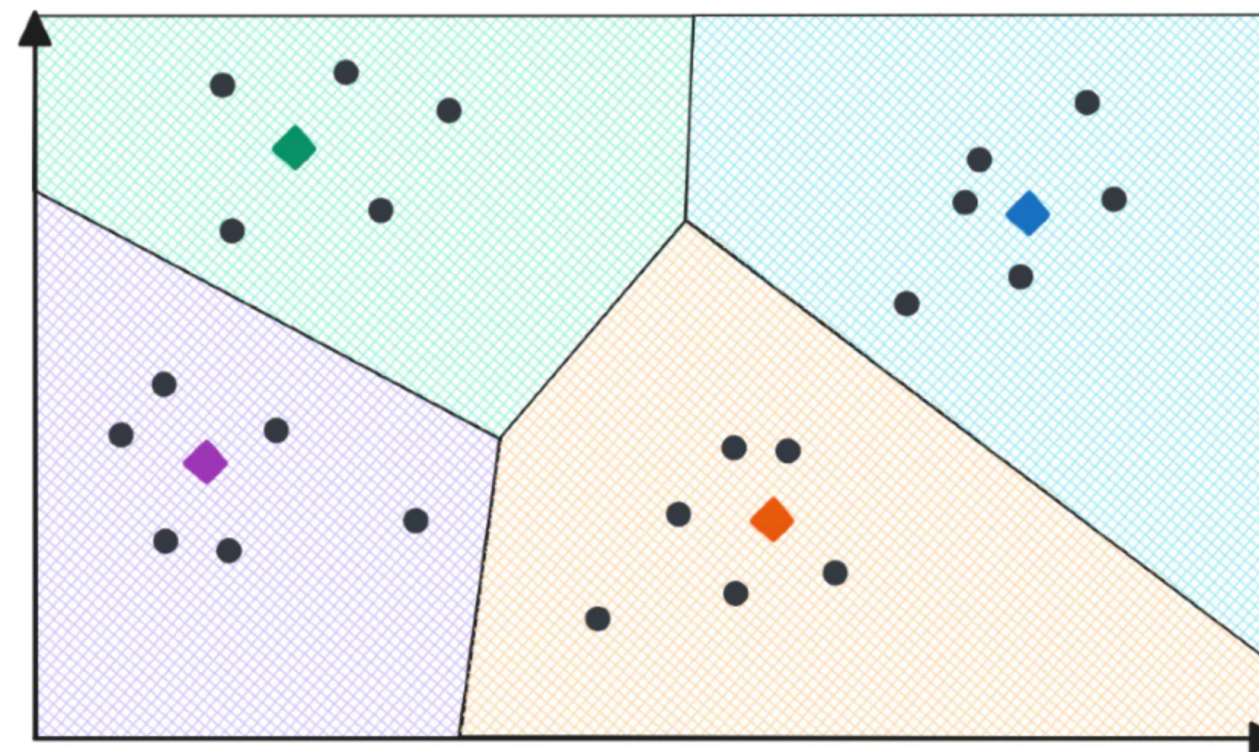
向量数据库常用检索算法

暴力搜索 (最近邻KNN: *K-NearestNeighbor*) 近似最近邻 (ANN: *Approximate Nearest Neighbor*)



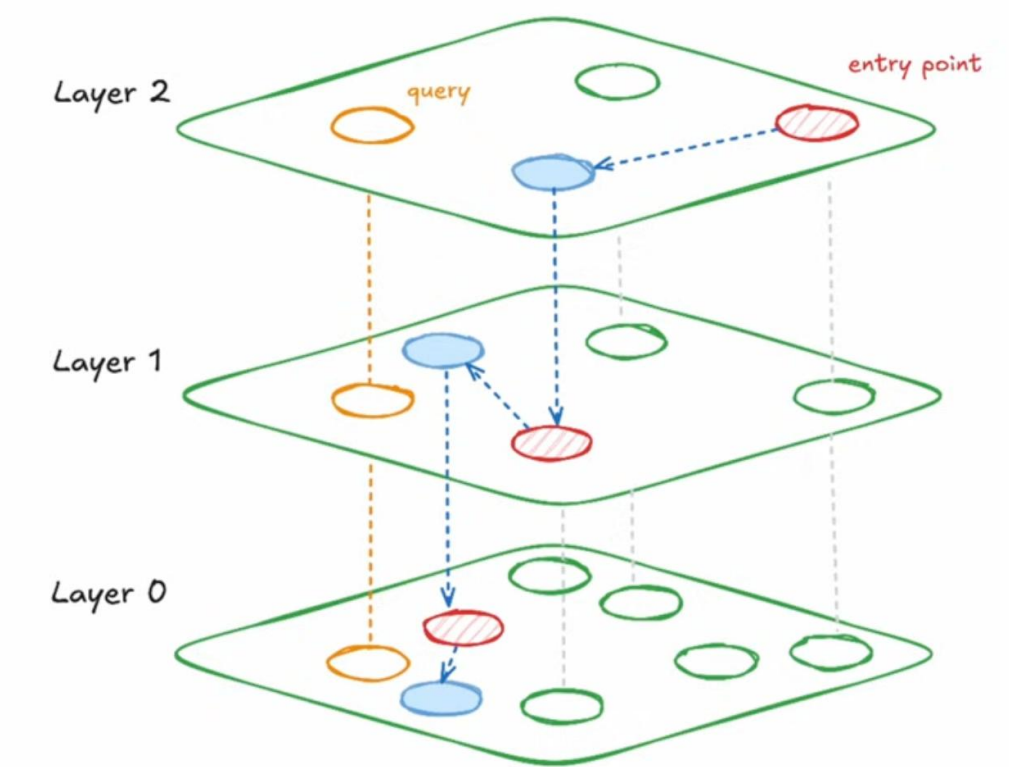
最近邻算法 (KNN) 通过计算查询点与数据库中每一个点的距离，找出最近的 k 个邻居。虽然结果精确，但在大规模数据集上速度极慢，不适用于实时应用。

IVF (倒排文件索引)



近似最近邻算法 (ANN) 在速度和精度之间做了权衡，不保证找到绝对最近的邻居，但能在极短的时间内找到非常接近的结果。在RAG检索等场景下，这种速度上的提升远比微小的准确性损失更重要。

HNSW (分层导航小世界)



向量数据库检索算法

最初的尝试: 暴力搜索 -- 精准但缓慢的“地毯式”排查



向量数据库检索算法

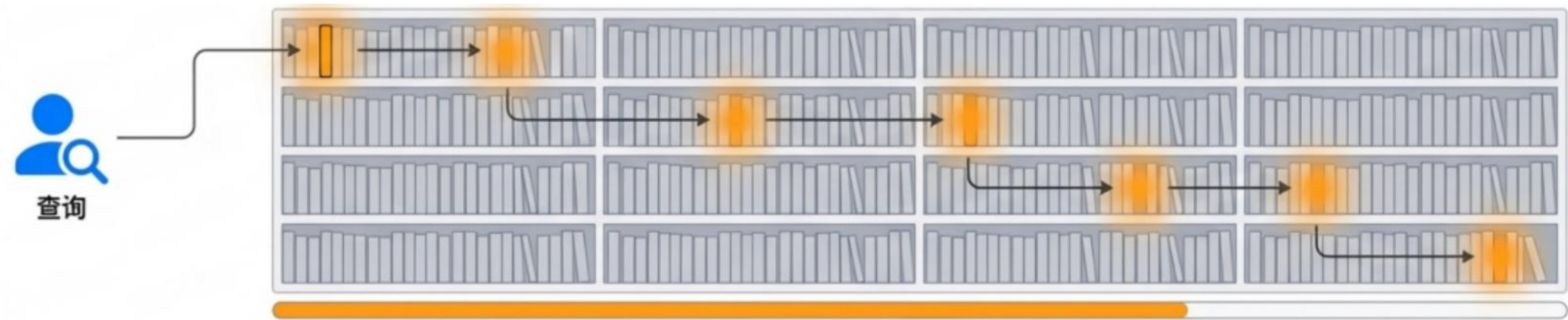
最初的尝试: **暴力搜索** -- 精准但缓慢的“地毯式”排查

工作原理:

- ✓ 接收一个查询向量。
- ✓ 逐一计算它与数据库中 每一个 向量的距离(如欧氏距离或余弦相似度)。
- ✓ 对所有计算出的距离进行排序，返回距离最小的前K个结果。

优点	缺点
精度100%，结果绝对准确	时间复杂度极高: $O(N)$ ，其中N是数据集中向量总数
实现简单，易于理解	不适用于大规模数据，在百万级数据集上响应时间可能是分钟甚至小时级别。

视觉化比喻：向量数据库中一个个排查



向量数据库检索算法

范式革命: 近似最近邻(ANN) -- 用可接受的精度换取极致的速度

与其追求100%精确的“最近邻 (KNN)”，不如在毫秒内找到99%相似的“近似最近邻 (ANN)”。对于大多数AI应用而言，这种速度上的提升远比微小的精度损失更有价值。ANN的核心是通过构建高效的数据索引结构，在搜索时避免对整个数据集进行扫描，从而大幅缩减搜索空间。



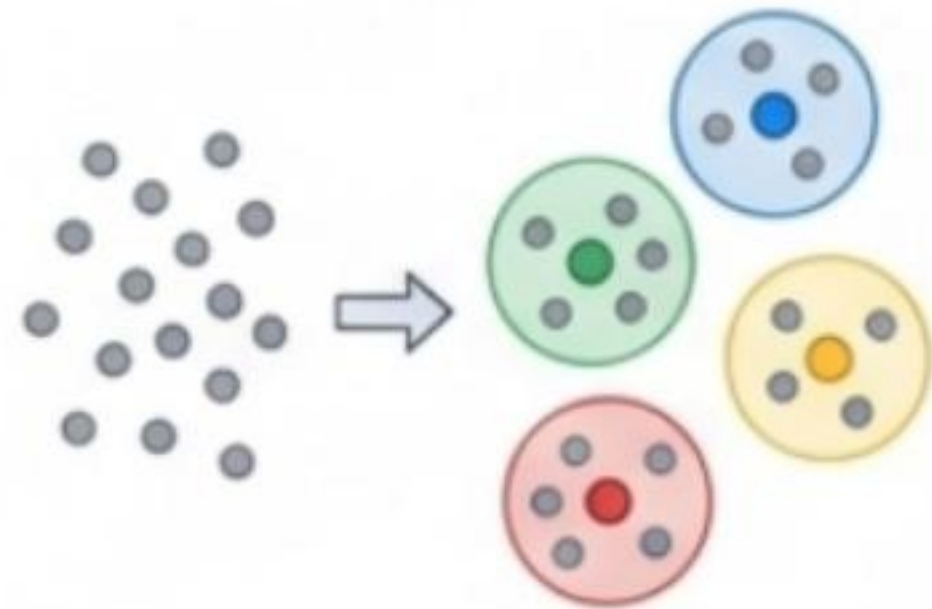
ANN算法的技术实现

让每个职场人都有定义AI的能力

向量数据库检索算法

主角登场: ① IVF: 先分区, 再精搜

IVF (Inverted File 倒排文件索引) 的工作流程



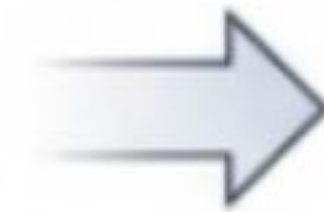
Step 1: 聚类(Clustering)

使用K-Means等算法将所有向量数据聚成 n_{list} 个簇(clusters)。每个簇的中心点称为质心(centroid)。



Step2: 索引(indexing)

创建一个“倒排文件”，即一个从“质心”到“该簇内所有向量”的映射列表。



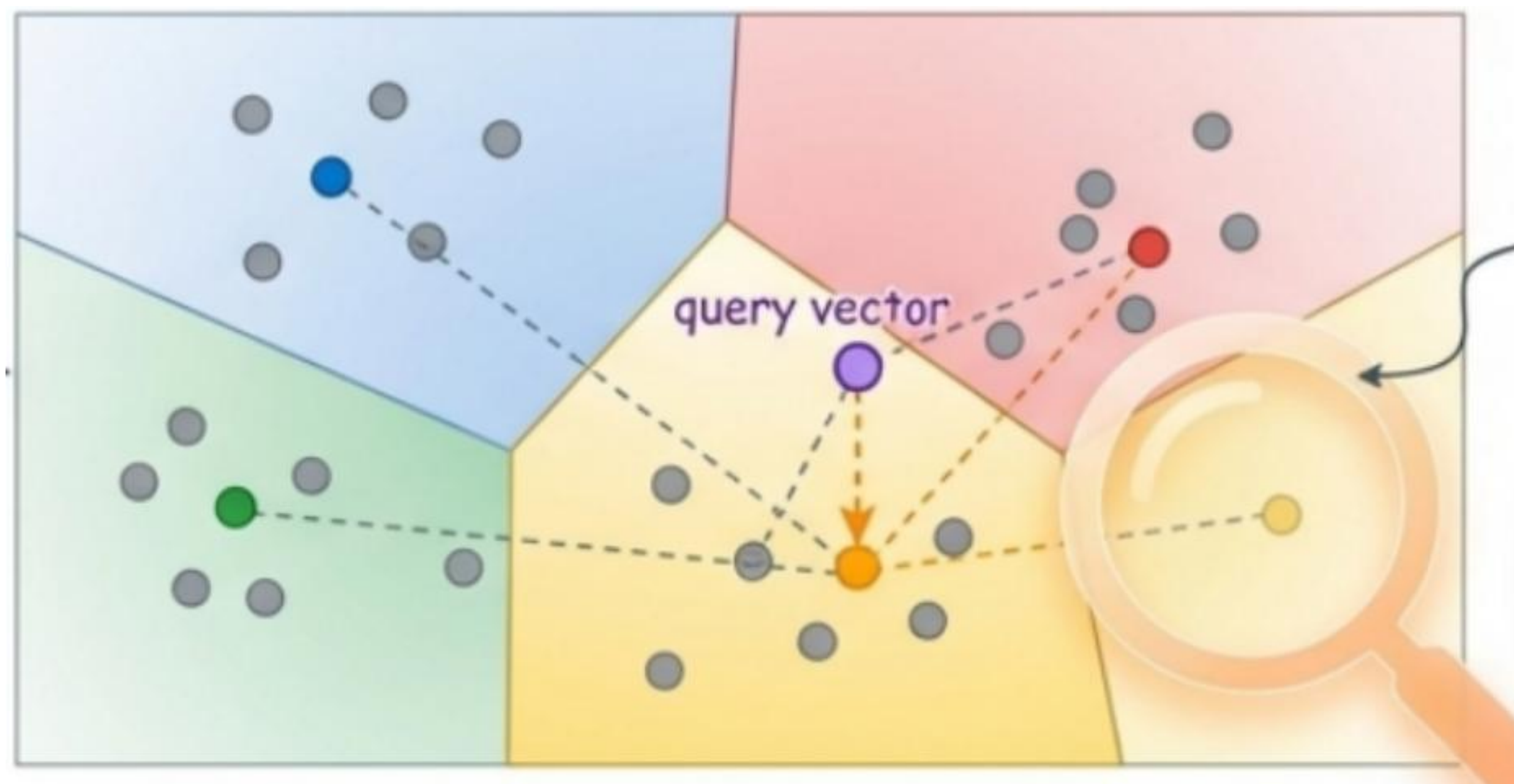
Step 3:查询(Querying)

计算查询向量与所有 n_{list} 个质心的距离。选出最近的 n_{probe} 个簇。仅在这 n_{probe} 个簇内进行暴力搜索，找到最终的K个近邻。

向量数据库检索算法

主角登场: ① IVF: 先分区, 再精搜

IVF (Inverted File 倒排文件索引) 的工作流程



① 将整个空间划分为多个区域（簇）。

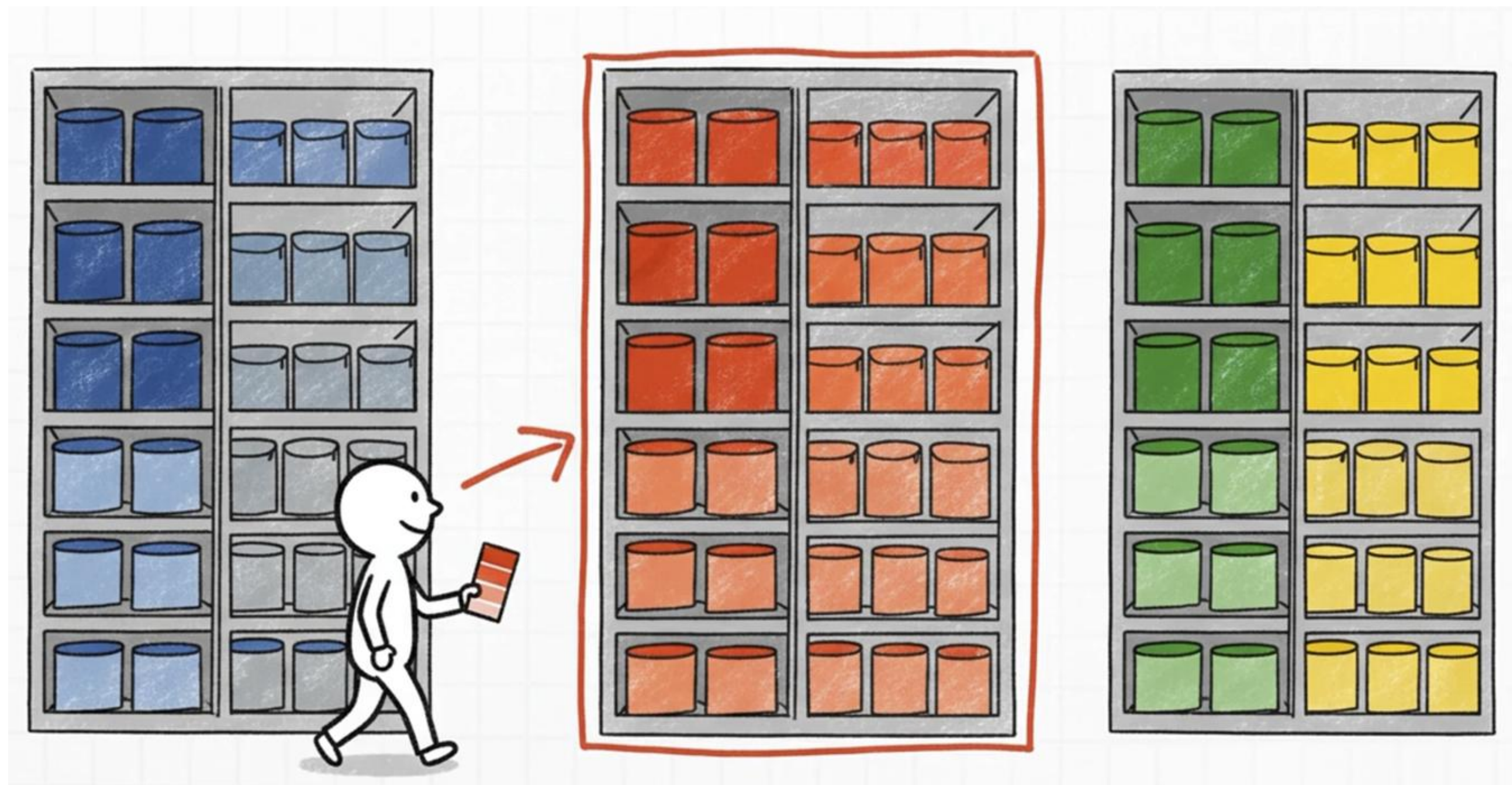
② 查询向量首先与各个区域质心比较，确定其归属的区域（黄色区域）。

③ 搜索query vector 所在的黄色区域及其邻近的少数几个区域内，大大减少了计算量

向量数据库检索算法

主角登场: ① IVF: 先分区, 再精搜

IVF (Inverted File 倒排文件索引) 的工作流程



- ① 将颜色划分为多个区域（簇）。
- ② 查询颜色首先与各个区域质心比较，确定其归属的区域（簇）。
- ③ 搜索所在的区域及其邻近的少数几个区域内。

向量数据库检索算法

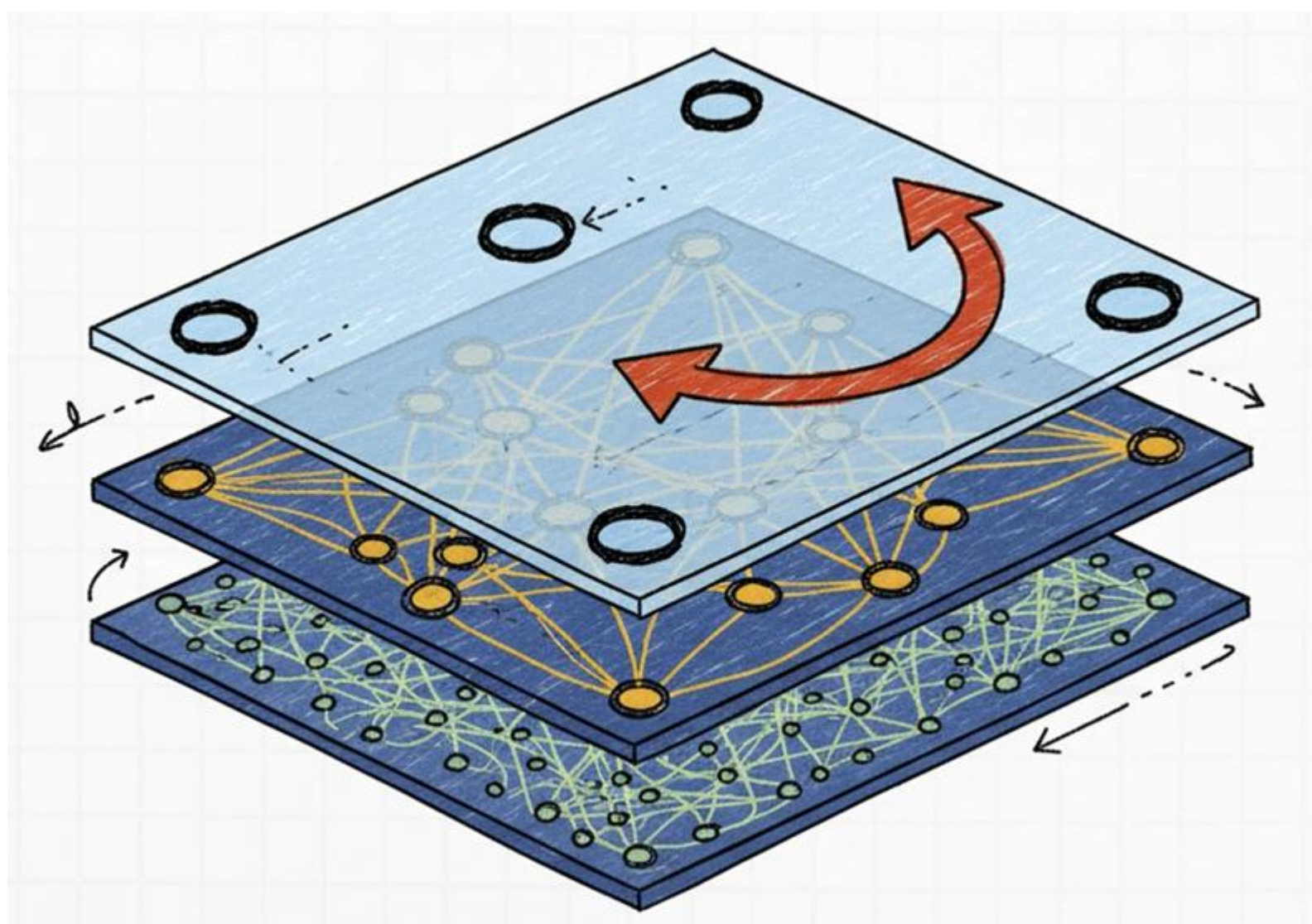
主角登场: ② HNSW (Hierarchical Navigable Small World Graph) : 分层导航小世界

HNSW的核心思想

HNSW(Hierarchical Navigable Small World)是一种基于图的ANN算法。它借鉴了“小世界网络”和“跳表”的思想。

小世界特性: 图中大部分节点不直接相连, 但任何两个节点之间都可以通过很少的几步 (“六度分隔”理论) 到达。

层次化结构: 构建多层图。高层图节点稀疏, 边连接距离远(比如: “高速公路”); 底层图节点密集, 边连接距离近(比如: “本地街道”)



向量数据库检索算法

主角登场: ② HNSW (Hierarchical Navigable Small World Graph) : 分层导航小世界

HNSW的搜索流程



进入最高层

从一个预设的入口点 (entry point) 开始



层内贪婪搜索

在当前层，遍历当前节点的邻居，选择离查询向量最近的邻居作为新的当前节点。重复此过程，直到找不到更近的邻居



向下移动

将当前层找到的最近点作为下一层的入口点。



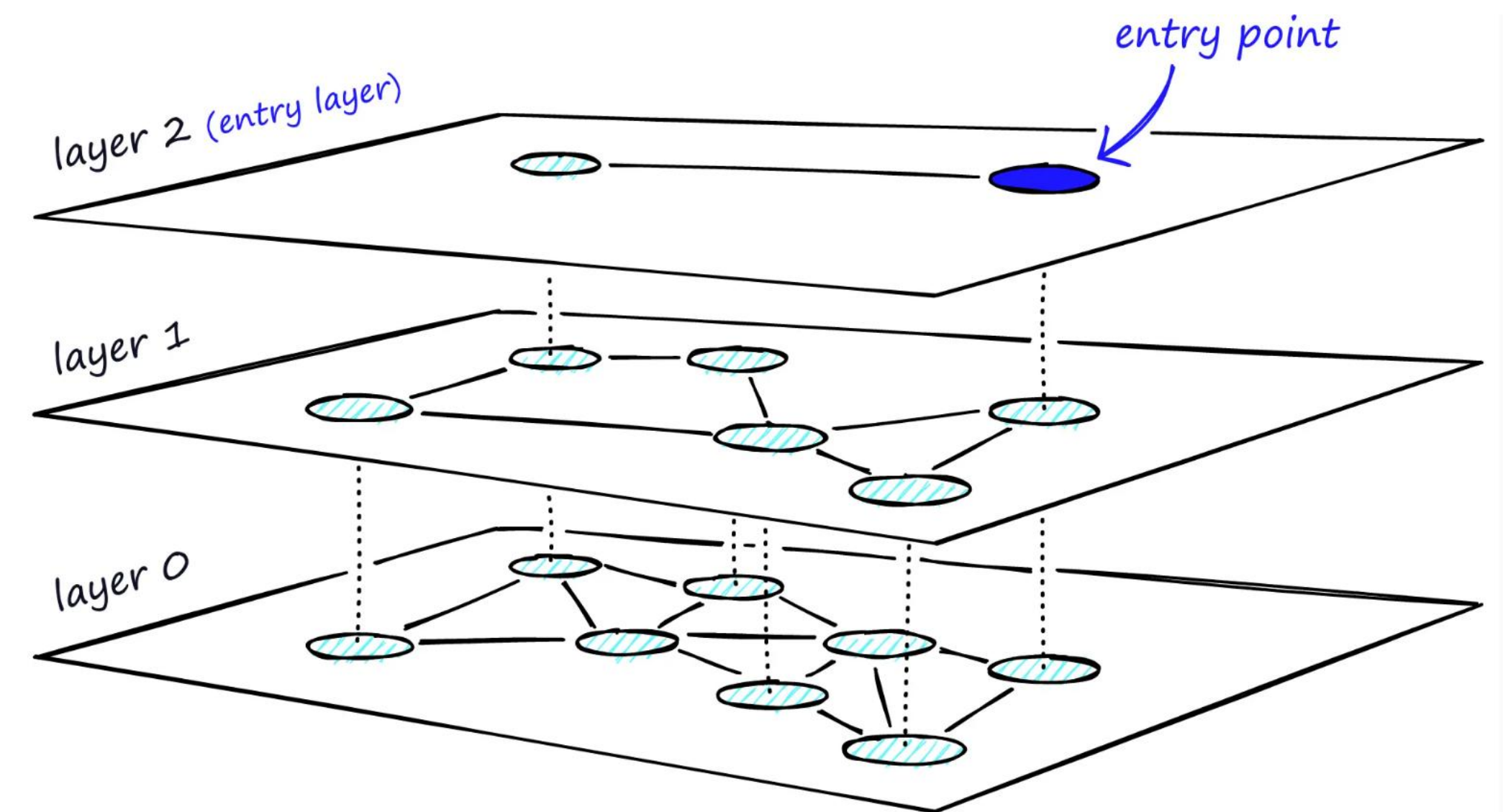
重复搜索

在下一层重复步骤2的贪婪搜索。



最终搜索

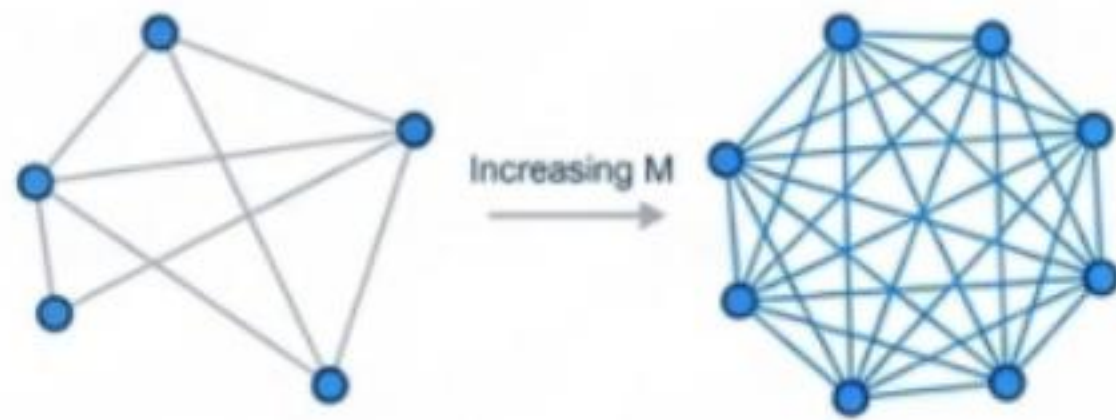
在最底层 (Layer0) 执行最终的精细搜索，返回K个最近邻。



调优你的HNSW引擎: 理解三大核心超参数

选择HNSW只是第一步。要发挥其最大潜力，必须理解并合理配置其核心超参数。这关乎在你的特定数据集上实现最佳佳的速度与精度平衡。

1. M（最大连接数）

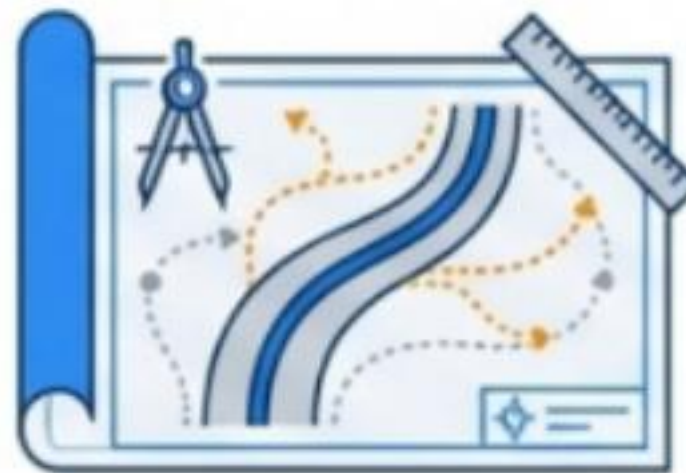


作用: 控制图中每个节点的邻居数量(出度)

影响: 较高的 M 值会创建一个更稠密的图通常能提高搜索质量(召回率)，但会显著增加内存使用和索引构建时间。

类比: 增加城市中每条街道的交叉路口数量。

2. efConstruction（构建时候选队列大小）

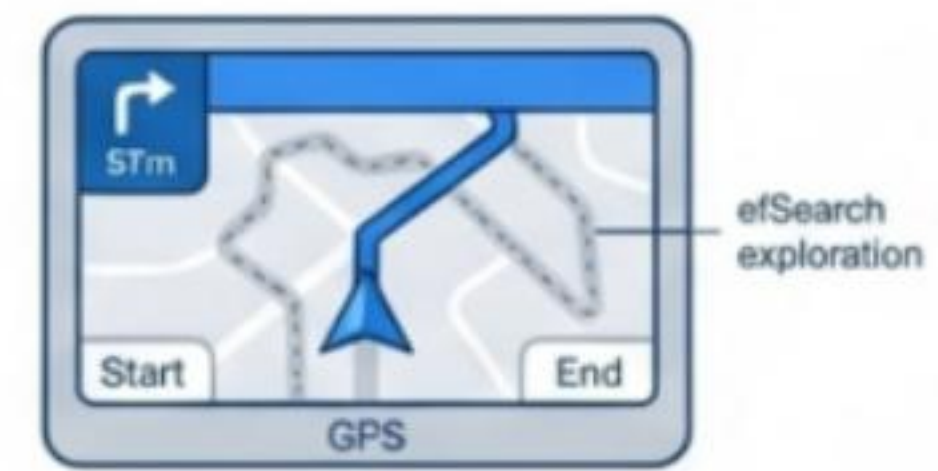


作用: 在构建索引期间，为每个新节点寻找邻居时维护的候选列表大小。

影响: 较高的 efConstruction 值可以构建出更高质量的图，有利于最终的搜索精度，但会大幅增加索引构建时间。

类比: 修道路时，考察更多的备选路线方案。

3. efSearch（搜索时候选队列大小）



作用: 在搜索期间，维护的动态候选列表大小。这是最重要的查询时参数。

影响: 较高的 efSearch 值会探索更广泛的路径，显著提高搜索精度(召回率)，但也会增加查询延迟。

类比: 导航时，同时考虑更多的备选路径而不是只走最短的那条。

巅峰对决: IVF 与 HNSW 的全方位对比

特性	IVF（倒排文件索引）	HNSW（分层可导航小世界）
搜索类型	基于聚类的搜索	基于图的搜索
搜索速度	快	非常快，尤其是在超大规模数据集的情况下
构建速度	索引构建速度相对较快	索引构建过程计算量大，相对耗时
内存占用	内存占用相对较低，尤其适合与量化技术结合，处理无法完全载入内存的超大规模数据集	索引需要将所有向量载入内存以获得高性能，内存开销巨大
准确率（召回率）	召回率和速度之间的权衡主要通过 <code>n_probes</code> 参数调节	通常能达到非常高的召回率，性能优越
动态数据	对新增数据不友好，通常需要定期重建索引	支持增量添加数据，无需完全重建索引，适合动态变化的数据集
适用场景	适合静态的、超大规模的数据集，对内存资源有限的场景非常友好	对查询性能和召回率要求极高的实时应用，且数据集可以完全加载到内存中